Name:Sameen Shahzad
Date:18/2/26

Fastapi Training

UV & Alembic

What is uv?

uv is a modern Python package manager and project manager.

It replaces:

- `pip`
- `venv`
- `pip-tools`
- `virtualenv`

It is:

- Extremely fast
- Creates isolated virtual environments
- Manages dependencies properly

Step to shift my existing project to uv

- STEP 1 — Freeze Your Current Dependencies

  Activate your OLD virtual environment:
  `venv\Scripts\activate`
  Then run:
  `pip freeze > requirements.txt`
  This saves all installed packages.

- STEP 2 — Deactivate and Remove Old venv

Deactivate: `deactivate`
delete your old venv folder manually:

- STEP 3 — Initialize uv in Your Project

Go to your project root: `cd your_project`
Run: `uv init`
Now uv creates:`pyproject.toml`

- STEP 4 — Create New uv Environment

Run: `uv venv`

It creates: `venv/`

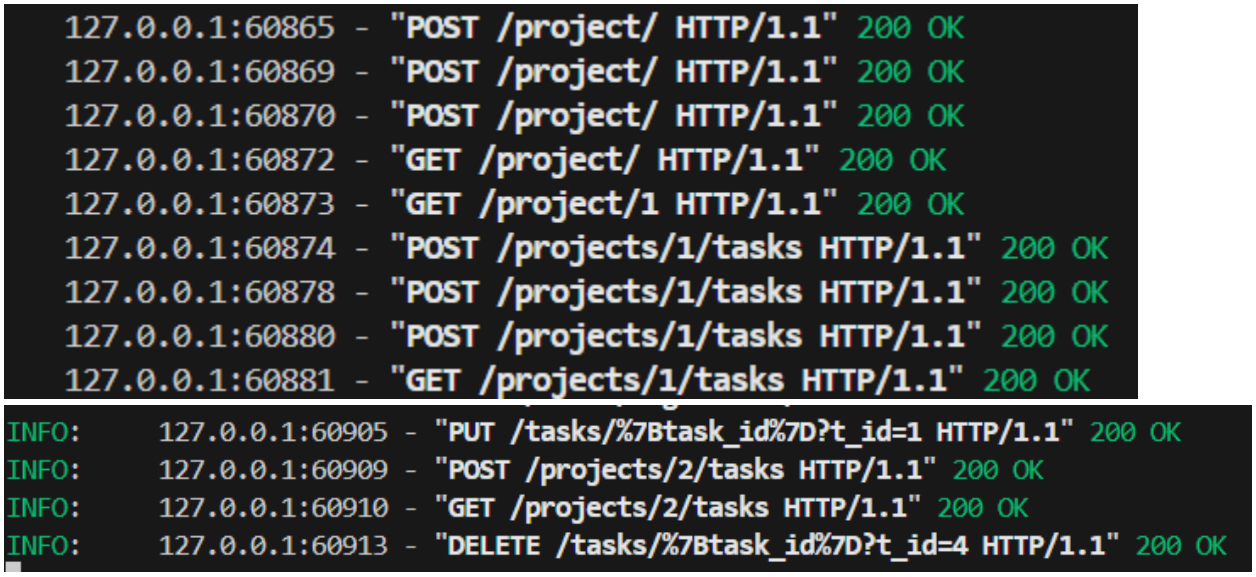
- STEP 5 — Install Old Dependencies into uv

Now install from requirements file:`uv add -r requirements.txt`
This will:

- Install all packages
- Add them to `pyproject.toml`
- Create lock file automatically

Instead of activating environment, just run:`uv run uvicorn main:app --reload`

Progress Update
Task 3: README updates — Completed
Task 2: Created handler folder and separated DB queries — Completed
Task 2: Added config folder
repos:https://github.com/SameenShahzad-rtc/backend_phase_1/tree/handler-config



What is Alembic?

- **Alembic** is a **database migration tool** for Python, primarily used with **SQLAlchemy**.
- **Database Migration Tool** means: software that **tracks, manages, and applies changes** to the database schema over time.

Key idea:
Your **Python code (models)** and your **actual database (tables)** can evolve separately. Alembic ensures **both are in sync safely**.

The workflow of Alembic is very similar to Git. When you make changes to your models, you create a new migration (like making a Git commit) using commands such as alembic **revision --autogenerate.** You then apply this migration to the database with alembic upgrade head, which is analogous to pushing or merging a commit in Git — it moves the database schema forward to match your models. If a mistake occurs or you need to revert a change, you can use **alembic downgrade**, just like checking out a previous commit in Git, which rolls back your database to a prior state. Alembic also keeps a full history of migrations, allowing teams to synchronize their databases safely across development, staging, and production environments.

Why Alembic is Needed

Problem without Alembic

Example:

```
# models.py
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String(50))
```

- You create the database → everything works.
- Later, you want to add a new column:

```
email = Column(String(100))
```

Without Alembic:

- Database still has only id and name.
- Running:

```
new_user = User(name="Ali", email="ali@example.com")
session.add(new_user)
session.commit()
```

→ Throws **OperationalError: no such column: email**

Conclusion: Python models do **not automatically update the database**.

How Alembic Solves It

- Alembic generates a **migration script**:

  ```
  ALTER TABLE users ADD COLUMN email VARCHAR(100);
  ```

- It will Apply it → database now matches the model.